

Keyloop Inventory Dashboard

An intelligent inventory dashboard that surfaces aging vehicle stock and lets dealership managers log follow-up actions against it — Scenario B of the Keyloop technical assessment.

SCENARIO	DOMAIN	LAYER BUILT	AUTHOR
B — Inventory Dashboard	Supply	Frontend	Akbar Basha Shaik

I work day-to-day in Keyloop's **Ownership** domain (Service Hub 360, Aftersales). I chose **Supply** deliberately — it tests my design process against a problem I don't already have domain familiarity with, rather than leaning on context from my current project.

01 Problem Summary

Dealership managers need a real-time view of vehicle stock that surfaces aging inventory — vehicles sitting for more than 90 days — and lets them record a follow-up action against each one, so stock that is losing value doesn't sit unmanaged.

02 Assumptions

The brief invites reasonable assumptions where requirements are ambiguous. Here are mine, and the reasoning behind each.

AMBIGUITY	ASSUMPTION	REASONING
Single or multi-dealership view?	Single dealership, no org/tenant switcher	Keeps scope matched to the time box; a tenant switcher is an additive UI change later, not an architectural one now
Is the "aging stock" threshold fixed?	Hardcoded >90 days, isolated in one pure function <code>isAgingStock</code>	One-line change to make it configurable later, without building config UI nobody asked for
Single current status, or a history?	Append-only action log per vehicle; current status = most recent entry	"Log" implies a timeline; an overwritable field would silently discard a manager's prior decisions
Vehicle data source	Static mock dataset with varied <code>daysInStock</code> values, served from a local REST mock	Matches the brief's instruction to mock the backend for a frontend submission
Authentication	Out of scope; single implicit manager identity	Not a stated requirement; adds surface area without demonstrating anything the brief evaluates

03 Architecture

The client is split into four layers so the business rule — what counts as aging stock — has exactly one home, and is testable without a browser. The API client is the single seam to the backend: swapping the mock for a real service later means changing a base URL, not any UI or domain code.

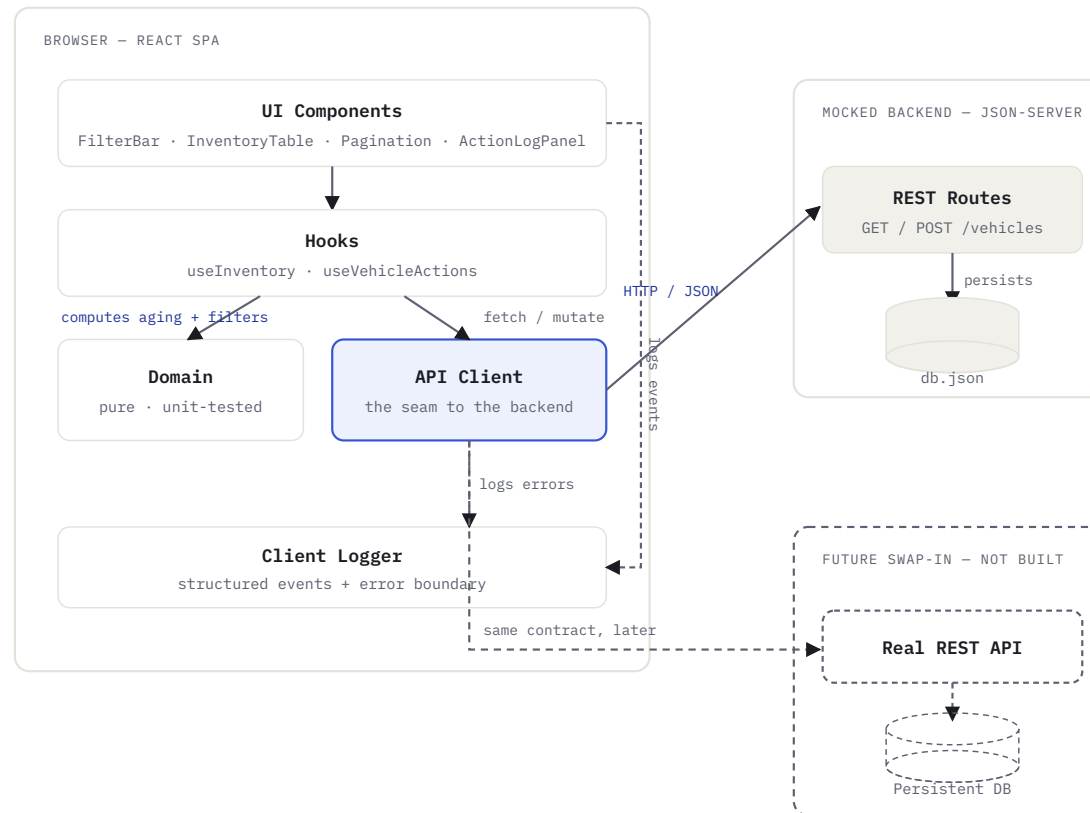


Figure 1 — The domain layer only ever sees data Hooks hands it, never the network; the API client is the one place that knows how to reach a backend, mocked today and real later.

04 Component Roles

COMPONENT	ROLE
UI Components	Presentation only. <code>InventoryTable</code> delegates the aging decision to the domain layer rather than computing it inline, so the rule has one home.
Hooks	Own data fetching, caching, and the loading/error state machine — the layer that would change if the app moved to a library like TanStack Query.
Domain	Pure, framework-free functions: the 90-day aging rule, filtering, sorting, pagination. No network, no React — this is what the test suite targets directly.
API Client	Thin fetch wrapper that attaches a correlation ID to every request and normalizes errors, so every caller gets a consistent error shape.
Client Logger	Structured event emission plus a React error boundary around the table and action panel, so one section failing doesn't blank the page.
json-server	A real persisted REST contract, not a static import — a posted action genuinely survives a refresh.

05 Data Flow

- **Load** — `InventoryPage` mounts → `useInventory()` calls `GET /vehicles` → response cached in hook state.
- **Aging check** — `isAgingStock(vehicle)` runs per row at render time, derived from `dateReceived` — not a stored flag — so there's one source of truth and no drift.
- **Filter** — `FilterBar` changes update local state; `filterVehicles()` recomputes the visible set client-side — no refetch needed for a single dealership's inventory. The result is sorted, then sliced into pages via `paginate()` ; a filter change resets the page back to 1.
- **Log an action** — manager submits a note in `ActionLogPanel` → `POST /vehicles/:id/actions` → json-server appends to `db.json` → UI reflects the new entry immediately.
- **Failure** — any network/mock failure is caught by the API client, normalized, logged, and surfaced as an inline error state rather than failing silently.

06 Requirements Coverage

Mapping each Scenario B acceptance criterion to where it's addressed, so coverage is verifiable at a glance.

REQUIREMENT	ADDRESSED BY
Inventory Visualization — filterable list	FilterBar + filterVehicles() , paginated via Pagination §4 / §5
Aging Stock Identification — flag >90 days	isAgingStock() , derived from dateReceived , rendered via AgingBadge §2 / §5
Actionable Insights — log/persist a status	ActionLogPanel → POST /vehicles/:id/actions → db.json §2 / §5

07 Technology Choices

CHOICE	WHY
React 18 + TypeScript + Vite	TypeScript catches drift between the UI and the mock API contract at compile time; Vite keeps the dev loop fast; React matches the stack already used on Service Hub 360.
json-server for the mock	A real HTTP + persistence round trip — a POST actually survives a refresh — exercises loading/error states more honestly than a static import.
Hooks, no global state library	One real piece of shared server state and simple local filter state. A Redux/Zustand layer would add indirection without solving a problem this app has.
Vitest + React Testing Library	Native pairing with Vite; RTL tests behavior rather than internals, fitting the aging-stock and filter rules the way a manager would encounter them.
Minimal CSS, no design system	A time-boxed exercise — a heavy UI kit spends budget on polish the evaluation criteria don't ask for, at the cost of implementation and tests, which they do.
Client-side pagination	A dealership's inventory fits in memory, so pagination here is a rendering concern, not a data-volume one — slicing an already-fetched array beats a round trip per page. The same paginate() call becomes a ?page=&pageSize= query param against a real API later with no UI change, since the component treats "the current page" as a prop, not "all the data."

08 Observability Strategy

The backend is mocked for this submission, so this states both what's implemented now and the production equivalent.

PILLAR	STATUS	APPROACH
Logging	implemented	Structured logger with event name + correlation ID + timestamp, for <code>inventory_loaded</code> , <code>filter_applied</code> , <code>action_logged</code> , <code>api_error</code> — the client-side half of the correlation pattern used with New Relic / App Insights on Service Hub 360.
Error isolation	implemented	React error boundary around the table and action panel; logs the failure before showing a fallback.
Metrics	documented	Production would report Web Vitals and a business metric — aging-stock count on load — to Azure App Insights, matching existing Keyloop tooling.
Tracing	documented	The correlation ID the API client already injects would propagate into the backend's distributed trace, so a support engineer can follow one action end to end.

09 GenAI Usage in the Design Phase

This was an AI-native design process — Claude Code was a core collaborator from the first draft, not an occasional lookup tool.

- **Ambiguity resolution** — I gave it the scenario requirements and had it catalog every place the spec was underspecified, more exhaustively than I'd have managed by hand. It surfaced the aging-threshold-configurability and single-status-vs-history questions above; I made the call on each and it helped draft the supporting reasoning.
- **Architecture & diagram drafting** — I described the component boundaries I wanted and it produced the full first-pass architecture, including the diagram. I reviewed it layer by layer: one early draft had the domain layer calling the API client directly, breaking the no-network-dependency property testability depends on, so that boundary was corrected.
- **Trade-off analysis** — I asked it to work through json-server vs. MSW vs. a static import, and React Query vs. plain hooks, with the real trade-offs for this app's scope. I accepted the plain-hooks call after confirming the reasoning held, and overruled a lean toward a global state library this app doesn't need.
- **Document drafting** — this document was written collaboratively: I set the structure and what each section had to satisfy, it wrote the first pass, and I edited for accuracy and rewrote sections where the reasoning needed sharpening.

The architecture reflects real AI-driven design work, directed and reviewed by me at every boundary — that direction-and-review loop, not withheld trust, is what shaped the final design.